

A* Search and Recursive Best-First Search

AIM:

- A) Implement the A* Search algorithm for solving a pathfinding problem.
- B) Implement the Recursive Best-First Search algorithm for the same problem.
- C) Compare the performance and effectiveness of both algorithms.

Code :

```
ai p2 T117.py - C:/Users/Adarsh/OneDrive/Videos/ai p2 T117.py (3.12.6)
File Edit Format Run Options Window Help
import heapq
import matplotlib.pyplot as plt

graph = {
    'MVLU College': {'Bandra': 12, 'Sion': 14},
    'Bandra': {'Worli': 6, 'Byculla': 10},
    'Sion': {'Byculla': 13},
    'Worli': {'Byculla': 5, 'CSMT': 9},
    'Byculla': {'CSMT': 6},
    'CSMT': {'Gateway of India': 3},
    'Gateway of India': {}
}

heuristics = {
    'MVLU College': 22,
    'Bandra': 14,
    'Sion': 13,
    'Worli': 9,
    'Byculla': 7,
    'CSMT': 2,
    'Gateway of India': 0
}

positions = {
    'MVLU College': (5, 8),
    'Bandra': (2, 6),
    'Sion': (8, 6),
    'Worli': (2, 4),
    'Byculla': (6, 4),
    'CSMT': (5, 2),
    'Gateway of India': (5, 0)
}
```

NAME : ADARSH SHUKLA

ROLL NO : T117

SUB : AI

A* Search and Recursive Best-First Search

```
ai p2 T117.py - C:/Users/Adarsh/OneDrive/Videos/ai p2 T117.py (3.12.0)
File Edit Format Run Options Window Help
    'Gateway of India': (5, 0)
}

def a_star(graph, heuristics, start, goal):

    queue = [(heuristics[start], 0, start, [start])]
    visited = set()

    while queue:

        f, g, current, path = heapq.heappop(queue)

        if current in visited:
            continue

        visited.add(current)

        print(
            "Visiting:", current,
            "| g =", g,
            "| h =", heuristics[current],
            "| f =", f
        )

        if current == goal:
            return path, g

        for neighbour, cost in graph[current].items():

            if neighbour not in visited:

                new_g = g + cost

                return path, g

        for neighbour, cost in graph[current].items():

            if neighbour not in visited:

                new_g = g + cost
                new_f = new_g + heuristics[neighbour]

                heapq.heappush(
                    queue,
                    (
                        new_f,
                        new_g,
                        neighbour,
                        path + [neighbour]
                    )
                )

        return None, float('inf')

start = 'MVLU College'
goal = 'Gateway of India'

path, distance = a_star(
    graph,
    heuristics,
    start,
    goal
)
```

NAME : ADARSH SHUKLA

ROLL NO : T117

SUB : AI

A* Search and Recursive Best-First Search

```
ai p2 T117.py - C:/Users/Adarsh/OneDrive/Videos/ai p2 T117.py (3.12.0)
File Edit Format Run Options Window Help
)

print("\nA* RESULT")
print("Path:", " -> ".join(path))
print("Distance:", distance, "km")

plt.figure(figsize=(12, 9))

for node in graph:
    x1, y1 = positions[node]
    for neighbour, weight in graph[node].items():
        x2, y2 = positions[neighbour]
        is_path = (
            (node, neighbour)
            in list(zip(path, path[1:]))
        )
        color = "orange" if is_path else "gray"
        width = 4 if is_path else 1.5

        plt.annotate(
            "",
            xy=(x2, y2),
            xytext=(x1, y1),
            arrowprops=dict(
                arrowstyle="->",
                color=color,

```

NAME : ADARSH SHUKLA

ROLL NO : T117

SUB : AI

Practical no. 2

A* Search and Recursive Best-First Search

```
ai p2 T117.py - C:/Users/Adarsh/OneDrive/Videos/ai p2 T117.py (3.12.6)
File Edit Format Run Options Window Help

    )

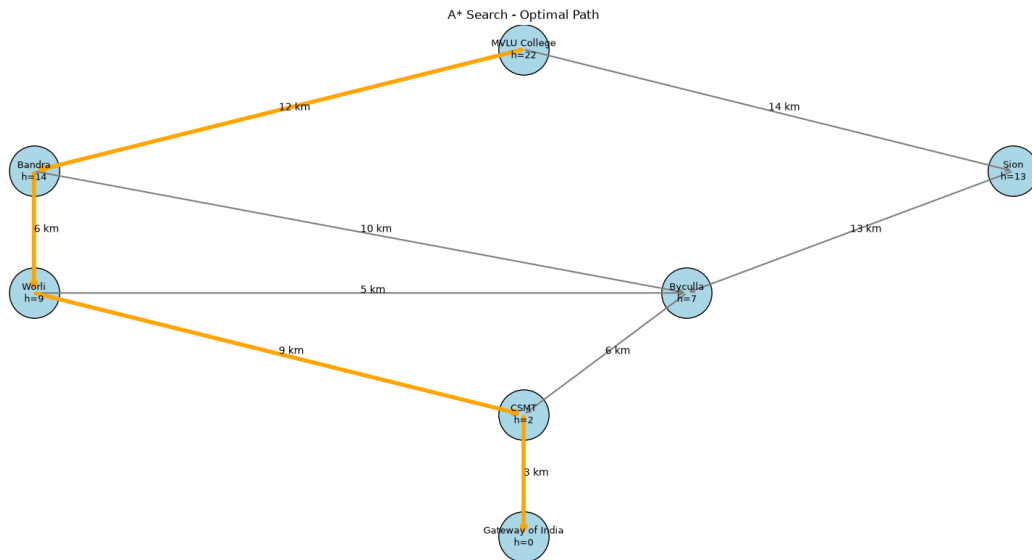
    plt.text(
        (x1 + x2) / 2,
        (y1 + y2) / 2,
        str(weight) + " km",
        fontsize=10
    )

for node, (x, y) in positions.items():

    plt.scatter(
        x,
        y,
        s=2500,
        color="lightblue",
        edgecolors="black"
    )

    plt.text(
        x,
        y,
        node + "\nh=" + str(heuristics[node]),
        ha="center",
        va="center",
        fontsize=9
    )

plt.title("A* Search - Optimal Path")
plt.axis("off")
plt.show()
```



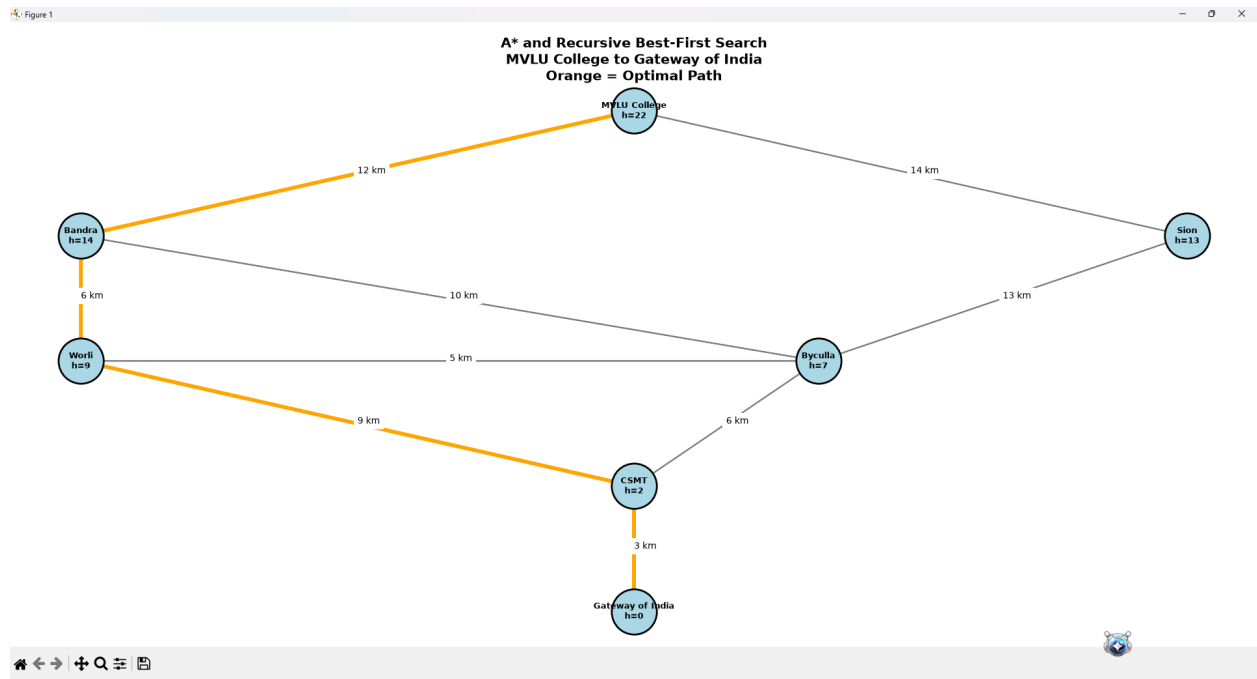
NAME : ADARSH SHUKLA

ROLL NO : T117

SUB : AI

Practical no. 2

A* Search and Recursive Best-First Search



Brfs

Code :

```

#BRFS
import matplotlib.pyplot as plt

graph = {
    'MVLU College': {'Bandra': 12, 'Sion': 14},
    'Bandra': {'Worli': 6, 'Byculla': 10},
    'Sion': {'Byculla': 13},
    'Worli': {'Byculla': 5, 'CSMT': 9},
    'Byculla': {'CSMT': 6},
    'CSMT': {'Gateway of India': 3},
    'Gateway of India': {}
}

heuristics = {
    'MVLU College': 22,
    'Bandra': 14,
    'Sion': 13,
    'Worli': 9,
    'Byculla': 7,
    'CSMT': 2,
    'Gateway of India': 0
}

positions = {
    'MVLU College': (5, 8),
    'Bandra': (2, 6),
    'Sion': (8, 6),
    'Worli': (2, 4),
    'Byculla': (6, 4),
    'CSMT': (5, 2),
    'Gateway of India': (5, 0)
}
    
```

NAME : ADARSH SHUKLA

ROLL NO : T117

SUB : AI

A* Search and Recursive Best-First Search

```
ai p2 T117.py - C:\Users\Adarsh\OneDrive\Videos\ai p2 T117.py (3.12.6)
File Edit Format Run Options Window Help

def rbfs(node, path, g, f_limit):

    print(
        "Visiting:", node,
        "| g =", g,
        "| h =", heuristics[node],
        "| f =", g + heuristics[node]
    )

    if node == goal:
        return path, g, True, g + heuristics[node]

    successors = []

    for neighbour, cost in graph[node].items():

        if neighbour not in path:

            new_g = g + cost
            new_f = new_g + heuristics[neighbour]

            successors.append(
                [
                    new_f,
                    neighbour,
                    path + [neighbour],
                    new_g
                ]
            )

    if not successors:
        return None, float('inf'), False, float('inf')

if not successors:
    return None, float('inf'), False, float('inf')

while True:

    successors.sort(key=lambda x: x[0])

    best = successors[0]

    if best[0] > f_limit:
        return None, float('inf'), False, best[0]

    if len(successors) > 1:
        alternative = successors[1][0]
    else:
        alternative = float('inf')

    result_path, result_cost, found, new_f = rbfs(
        best[1],
        best[2],
        best[3],
        min(f_limit, alternative)
    )

    best[0] = new_f

    if found:
        return result_path, result_cost, True, new_f

path, cost, found, final_f = rbfs(
    start,
    [start],
```

NAME : ADARSH SHUKLA

ROLL NO : T117

SUB : AI

A* Search and Recursive Best-First Search

```
ai p2 T117.py - C:\Users\Adarsh\OneDrive\Videos\ai p2 T117.py (3.12.6)
File Edit Format Run Options Window Help

    start,
    [start],
    0,
    float('inf')
)

return path, cost

start = 'MVLU College'
goal = 'Gateway of India'

path, distance = rbfs_search(
    graph,
    heuristics,
    start,
    goal
)

print("\nRBFS RESULT")
print("Path:", " -> ".join(path))
print("Distance:", distance, "km")

plt.figure(figsize=(12, 9))

for node in graph:
    x1, y1 = positions[node]

    for neighbour, weight in graph[node].items():

        x2, y2 = positions[neighbour]

        is_path = (
            (node, neighbour)
            in list(zip(path, path[1:]))
        )

        color = "green" if is_path else "gray"
        width = 4 if is_path else 1.5

        plt.annotate(
            "",
            xy=(x2, y2),
            xytext=(x1, y1),
            arrowprops=dict(
                arrowstyle="->",
                color=color,
                lw=width
            )
        )

        plt.text(
            (x1 + x2) / 2,
            (y1 + y2) / 2,
            str(weight) + " km",
            fontsize=10
        )

for node, (x, y) in positions.items():
```

NAME : ADARSH SHUKLA

ROLL NO : T117

SUB : AI

A* Search and Recursive Best-First Search

```

ai p2 T117.py - C:/Users/Adarsh/OneDrive/Videos/ai p2 T117.py (3.12.6)
File Edit Format Run Options Window Help

    )

    plt.text(
        (x1 + x2) / 2,
        (y1 + y2) / 2,
        str(weight) + " km",
        fontsize=10
    )

for node, (x, y) in positions.items():

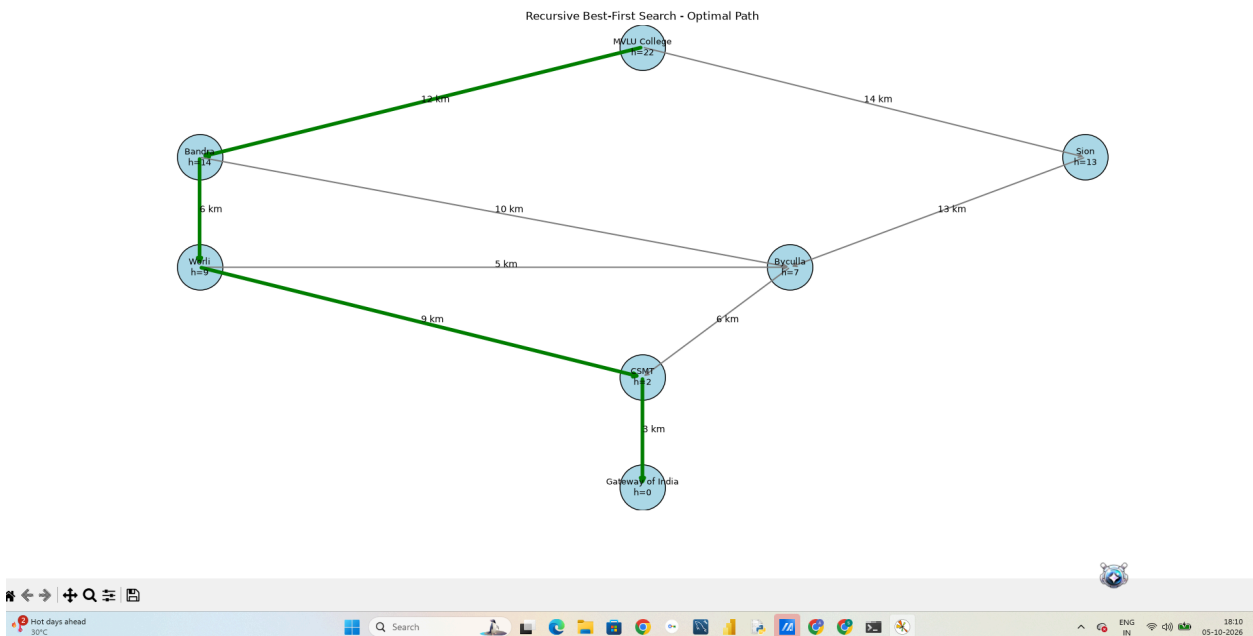
    plt.scatter(
        x,
        y,
        s=2500,
        color="lightblue",
        edgecolors="black"
    )

    plt.text(
        x,
        y,
        node + "\nh=" + str(heuristics[node]),
        ha="center",
        va="center",
        fontsize=9
    )

plt.title("Recursive Best-First Search - Optimal Path")
plt.axis("off")
plt.show()

```

Output :



NAME : ADARSH SHUKLA

ROLL NO : T117

SUB : AI